NLP Search Engine

A small, hackable information−retrieval engine in pure Python

Hamza Abdul Karim · F23607046 — May 2026

ABSTRACT

This project implements the classical information-retrieval stack — tokenization, Porter stemming, an inverted index, and TF-IDF / Okapi BM25 ranking — in roughly two thousand lines of Python. The code is organised into single-responsibility files of fewer than two hundred lines each, so it can be read in one sitting. Two front-ends share the same SearchEngine facade: a command-line interface and a Flask web application that also exposes a JSON API. The system was built as an NLP coursework project and is also a usable search engine for small corpora.

1. What's Inside

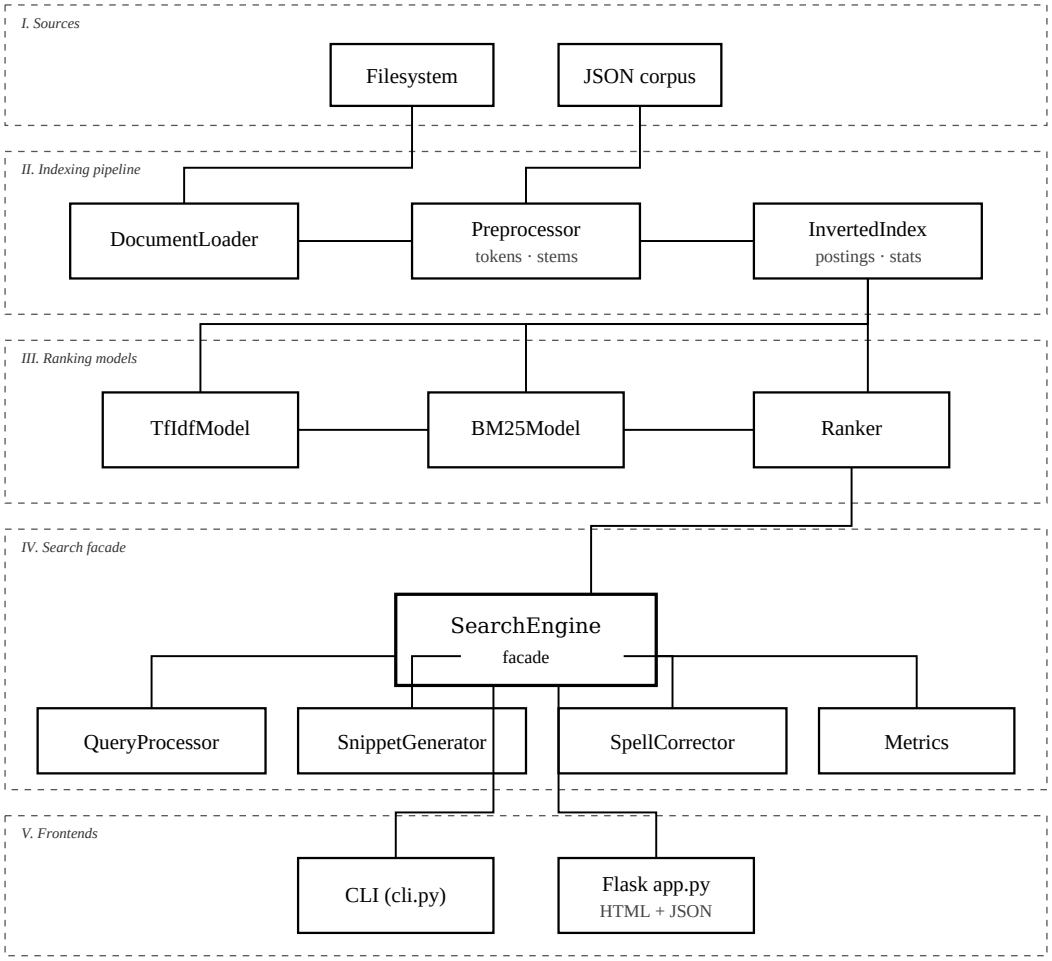
- NLP preprocessing — tokenization, case folding, stop-word removal, and a compact Porter stemmer.
- Inverted index — postings carry term frequencies and positions, enabling phrase queries.
- TF−IDF and BM25 — two interchangeable rankers; BM25 has tunable k1 and b parameters.
- Snippets and highlights — best-window selection with matched-term highlighting.
- Spell suggestions — Damerau–Levenshtein matching against the index vocabulary.
- Two front-ends — argparse CLI and a Flask web application with a JSON API at `/api/search`.

2. Architecture at a Glance

The system is organised into five horizontal layers, shown in Figure 1. Inputs flow into the indexing pipeline; the resulting inverted index is consumed by two interchangeable ranking models, exposed through a thin Ranker facade. The SearchEngine ties everything together and is the single entry point for both the CLI and the web application. The full set of architectural diagrams is in `docs/architecture.pdf`.

Figure 1. System architecture

Components and their compile-time dependencies



Solid arrows denote a "depends on" relation. Dashed boxes group components by responsibility.

Figure 1. System architecture (see docs/architecture.pdf for the full set).

3. Quickstart

```
git clone https://github.com/AbdulGhani002/nlp-search-engine.git
cd nlp-search-engine

pip install -r requirements.txt
python scripts/build_index.py

python cli.py "machine learning" # one-shot CLI
python cli.py # CLI in REPL mode
python app.py # Flask UI on :5000
```

Or with Docker:

```
docker build -t nlp-search-engine .
docker run --rm -p 5000:5000 nlp-search-engine
```

4. Demo Queries

```
information retrieval
"natural language"
ranking +bm25 -neural
neural NOT toy
```

5. Project Layout

```
nlp-search-engine/
app.py Flask web UI
cli.py Argparse CLI
pyproject.toml Packaging + tool configuration
Dockerfile Production container
Makefile Common commands
data/sample_docs/ Tiny NLP-themed corpus
docs/ PDFs and SVG diagrams
examples/ Standalone usage examples
scripts/ build_index.py, build_docs.py
src/ Engine source (every file <200 lines)
static/ CSS bundle and SVG brand assets
templates/ Jinja2 templates for the web UI
tests/ Pytest suite
```

6. Documentation

Topic	Location
System overview and diagrams	docs/architecture.pdf
TF-IDF and BM25 in depth	docs/ranking-models.pdf
HTTP/JSON API	docs/api.pdf
Dev environment and contributing	docs/development.pdf and CONTRIBUTING.pdf
Release history	CHANGELOG.pdf

7. Testing

```
pytest -q
```

The suite is fast (well under a second) and covers preprocessing, the inverted index, both rankers, the query parser, snippets, spell suggestions, and the IR metric helpers.

8. License